CAPSTONE PROJECT 02

BookFinder: Library Book Finder with Natural Language Search

The Sakura City Library network: 5 branches in Bengaluru and Tokyo, 8,000 titles in English and Japanese.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

1. The customer's problem

Members find the catalogue search frustrating: it needs exact titles. Librarians spend hours answering "I want something like..." questions. The library wants natural-language search, explainable recommendations and live availability across branches.

2. Users

- Library members
- Librarians at the help desk
- Branch managers planning purchases

3. User stories

- As a member, I search "a gentle mystery set in Kyoto" and get relevant books with a one-line reason each.
- As a member, I see which branch has a copy available now and can reserve it.
- As a member, I get "if you liked this" suggestions based on my loans.
- As a librarian, I see searches that returned nothing, so I know what to buy.
- As a branch manager, I see loan trends by subject and language.

4. Requirements

Must have

- Catalogue and loans data model in SQL (books, copies, branches, members, loans) with SQLAlchemy relationships
- Hybrid search (BM25 + embeddings) over title, author, summary and subjects with filters (language, branch, available now)
- Recommendation explanations generated from catalogue data with citations to book ids, no invented books
- Reservation flow with transaction safety (no double booking)
- React UI: search, results with availability, book page, my loans
- Evaluation: search hit rate@5 on a golden set of 25 natural-language queries

Should have

- Japanese and English queries (language slice in evaluation)
- Zero-result search report for librarians
- Re-ranking step

Could have (stretch)

- Reading lists generated for themes
- Accessibility features (large text, screen reader labels)
- Weekly drift report on search topics

5. Data

Starter notebook generates 8,000 books (title, author, language, subjects, summary, year), copies per branch, members and 40,000 loans.

6. Suggested AI approach

Embeddings and hybrid search with RRF (Day 23); grounded explanations (Day 22); optional query rewriting (Day 26).

7. Suggested architecture

```
React UI -> FastAPI back end -> database (SQL / MongoDB)
              |-> AI service (retrieval, model, tools, guardrails)
              |-> logs and metrics -> dashboard / weekly report
CI/CD: tests + quality gate -> Docker images
       -> Azure (Container Apps or App Service)
```

8. How to do it: day by day

Day 36 (Mon Dec 14): Kick-off and design

- Quiz on Day 35
- Teams of 3 to 4 formed; project chosen
- Problem statement, users, success metrics, assumptions log
- Architecture diagram and data model

Day 37 (Tue Dec 15): Core build

- Stand-up (15 min)
- Data loaded; back end API with main endpoints
- AI core working end to end (retrieval, recommendation, agent or pipeline)
- Golden set draft of 20 items

Day 38 (Wed Dec 16): Complete features and quality

- Stand-up
- Front end connected to the API

- Guardrails and error handling
- Evaluation run and first error analysis

Day 39 (Thu Dec 17): Harden, deploy, observe

- Stand-up
- Docker images; deploy to Azure or a documented local deployment
- Structured logs and a monitoring view
- Red-team and slice checks; fix top issues

Day 40 (Fri Dec 18): Demo day

- Final rehearsal (morning)
- Demo: 10 minutes + 5 minutes Q&A; per team
- Evaluator scoring with the rubric
- Individual reflection and peer feedback

9. Project-specific acceptance checks

Evaluators run these during or after the demo. They count towards Core functionality and AI quality.

- Search beats keyword-only on hit rate@5
- Every recommended book exists in the catalogue
- Reservation is atomic under concurrent requests
- Japanese queries evaluated separately

10. Golden set: starter items

Extend these to at least 20 items (30 for PolicyPal) with expected answers, sources and tags.

- "historical fiction about samurai" must return books with the subject historical fiction and Japan
- "books by Haruki" must not return invented titles
- "something for a 10 year old about space" must filter to children's science

11. How you will be evaluated

Criterion	Points	For this project, evaluators look for
Problem understanding and scoping	10	Clear problem statement, users, success metrics, assumptions log, agreed MVP scope (Days 15, 20).
Architecture and design	10	Sensible architecture diagram, layered code, right tool choices with reasons, data model (Days 4, 6, 7, 22, 33).
Core functionality	20	All must-have features work end to end in the demo; project checklist items delivered.
AI and data quality	15	Golden set of at least 20 items, measured metrics (retrieval, faithfulness or recommendation quality), error analysis (Days 23, 29, 30).
Engineering quality	10	Readable code, tests (API, pipeline, prompts), Git history with pull requests, CI pipeline green (Days 13, 17, 18).
Security, guardrails and responsible AI	10	Input and output guardrails, secrets handling, least privilege, a red-team check, fairness or language slice check (Days 8, 28).
Deployment and operations	10	Containerised, deployable (Azure or documented), structured logs, a dashboard or report, one drift or quality check (Days 19, 31 to 35).
Demo and communication	10	Answer-first readout, live demo, honest limits, confident Q&A;, clear handover README (Days 15, 20, 24).
Teamwork and process	5	Daily stand-ups, decision log, fair contribution, kind and specific feedback (Days 15, 24, 31).

12. Common pitfalls

- Generating book titles with the model
- Ignoring language in evaluation
- No transaction on reservations

13. Getting started

- Open Starter_Notebooks/Capstone_02_BookFinder_Starter.ipynb in Colab or VS Code to generate the data and see a working baseline.
- Use App_Build_Prompts/Capstone_02_BookFinder_Prompts.md with your AI coding assistant to scaffold the project.
- Write your problem statement and assumptions log before writing code.

Ganbatte (頑張って): do your best